Sorting & Searching

Asten Valentinus

Sorting and searching are *both* valuable types of algorithm for a programmer to have under her belt.

If you are—for example—implementing your own custom `Dictionary` class, you'd need a searching algorithm to scan through keys. Getting an element from a dictionary isn't free like it is for a pointer-array-style list.

The finished code is available here, if you want to skip ahead.

Here's the first iteration of the class, without any of the fancy sorting & searching algorithms:

```
"""An implementation of an unordered dictionary."""

from typing import Any


class UnorderedDictionary[ElementType = Any]:
    """An implementation of an unordered dictionary.

    Created to demonstrate various sorting and searching algorithms.
    """

    def __init__(self, keys: list[str], values: list[ElementType]) -> None:
        """Initialize using raw `keys` and `values` lists.

        Args:
            keys: Keys of the elements.
            values: Values of each elements.

        Raises:
            ValueError:
                If the length of the `keys` and `values` lists do not match.
        """
```

```
        if len(keys) != len(values):
            msg: str = "Length of `keys` and `values` lists does not match."
            raise ValueError(msg)

        self.keys = keys
        self.values = values

    @staticmethod
    def from_dict(
        base_dict: dict[str, ElementType],
    ) -> UnorderedDictionary[ElementType]:
        """Create an `UnorderedDictionary` based on a Python `dict`.

        Args:
            base_dict: The `dict` to get the keys and values from.

        Returns:
            A `UnorderedDictionary` with the same keys and values as the
            provided `dict`.
        """
        return UnorderedDictionary(
            list(base_dict.keys()),
            list(base_dict.values()),
        )

    def get_by_linear_search(self, key: str) -> ElementType | None:
        """Get an element of the dictionary using a linear search algorithm.

        This is the simplest (and most inefficient) searching algorithm.
        It simply loops over the keys until it finds the one we're looking for.

        Args:
            key: The key associated with the value to get.

        Returns:
            The value associated with the key, or `None` if the key is
            not defined.
        """
        found_index: int = -1

        for i, key_i in enumerate(self.keys):
            if key_i == key:
```

```
                found_index = i
                break

        if found_index == -1:
            return None

        return self.values[found_index]
```

The linear search—though simple to implement—can be quite inefficient for larger data-structures. The Big-O notation is $O(n)$, meaning that in the worst-case scenario, this takes an amount of time proportional to the size of the dictionary.

Better is possible. In the next section, I'll show the *Binary Search* algorithm.

Binary Search

The *Binary Search* algorithm brings the time-complexity down to $O(\log_2 n)$. That's a huge improvement, and only saves more as the dictionary size increases.

There's a downside however; the dictionary must be sorted by the keys.

Here's how it might go:

1. Pick the key *right* in the middle of the sorted dictionary.
2. Check whether that key is above or if it's below the key we're looking for. This is possible because the keys are sorted (i.e. lexicographically)
3. If it's above, start again at 3/4 through the dict, if it's below, 1/4.

It's like playing a number guessing game! With this algorithm, it narrows down the possibility by huge amounts at a time.

It can be implemented like this:

```
def get_by_binary_search(self, key: str) -> _GetterReturnType:
    """Get an element of the dictionary using the Binary Search algorithm.

    The dictionary is expected to be sorted lexicographically before
    execution of this method. Most of the time (unreliably), this method
    will incorrectly return `None` when the dictionary isn't sorted.

    Args:
        key: The key associated with the value to get.
```

```
    Returns:
        The value associated with the key, or `None` if the key is
        not defined.
    """
    # Signifies the range that may contain the key we want.
    search_range: tuple[int, int] = (0, len(self.keys) - 1)

    while True:
        checked_key_index = int((search_range[0] + search_range[1]) / 2)
        checked_key: str = self.keys[checked_key_index]

        if checked_key < key:
            search_range = (checked_key_index + 1, search_range[1])

        elif checked_key > key:
            search_range = (search_range[0], checked_key_index - 1)

        elif checked_key == key:
            return self.values[checked_key_index], checked_key_index

        # This always eventually is true when the key is undefined.
        if search_range[0] > search_range[1]:
            raise KeyError
```

Sorting

As mentioned above, the dictionary will not work correctly if it isn't sorted.

How do we ensure it's properly sorted? There are several algorithms at our disposal.

Many high-level programming languages/libraries implement these for us, including Python. For those that don't however, and for cases where existing interfaces aren't applicable, it is important to have these in our mental toolbox.

Here I'll show the implementations of a few different sorting algorithms and their Big-O analyses.

Bubble Sort

Bubble sort (or 'sinking sort') is likely the easiest sorting algorithm to understand and implement, but is quite inefficient. While learning sorting algorithms, it can be the best introductory option.

Here's how it works:

```
def sort_by_bubble(self) -> None:
    """Sort the dictionary using the Bubble Sort algorithm.

    Based on <https://www.geeksforgeeks.org/dsa/bubble-sort-algorithm/>.
    """
    # Loop over each element, ensuring each one is in the
    # correct place.
    for i in range(self._len):
        swapped = False

        # Loop over each element *after* element `i`, as `i` and before
        # are already in place.
        for j in range(self._len - i - 1):
            # If the key isn't in order with its successor, swap them.
            # The greater-than operator works on strings, going by
            # lexicographical order.
            if self.keys[j] > self.keys[j + 1]:
                # Swap the key with its successor.
                self.keys[j], self.keys[j + 1] = (
                    self.keys[j + 1],
                    self.keys[j],
                )
                # Swap the value with its successor, to bring them into
                # order relative to eachother.
                self.values[j], self.values[j + 1] = (
                    self.values[j + 1],
                    self.values[j],
                )
                # Track that this operation happened (see below).
                swapped = True

        # If the inner loop didn't cause any sorting, this means no
        # more sorting is necessary.
        if not swapped:
            break

    self._sorted = True  # Used to track whether sorting is needed when
    #                      performing operations that require it to be.
    #
    #                      It was added to __init__, defaulting to `False`.
```

With that sorting algorithm, we can now ensure the dictionary is sorted before binary searching.

I've removed the note from the docstring about sorting, and added the following snippet to the start of `get_by_binary_search`:

```
if not self._sorted:
    self.sort()  # A wrapper function, pointing to a default sort algorithm.
```

Currently, `sort` is

Given all that, we can now run a `binary_search` without having to manually sort it!:

```
from unordered_dictionary import UnorderedDictionary

example_dictionary = UnorderedDictionary.from_dict(
    {
        "A": 1,
        "B": 2,
        "C": 4,
        "D": 9,
    }
)

print(f"A: {example_dictionary.get_by_binary_search('A')[0]}")
print(f"C: {example_dictionary.get_by_binary_search('C')[0]}")
```

```
A: 1
C: 4
```

As we can see, everything is functioning as expected; A is 1, and C is 4.

Insertion Sort

Quick-Merge Sort

Data-Structure Traversal